

INTRODUCTION TO DATA SCIENCE ASSIGNMENT

Step 1: Loading Data

Read the Iowa data file into a Pandas DataFrame called home_data.

The screenshot shows the PyCharm IDE with a file explorer on the left displaying a project structure with files: iowaHousing.csv, target.csv, test.csv, train.csv, and iowa. The main editor shows a Python script named ReadingFromData.py with the following code:

```
4 # WE Define the path to your data file
5 iowa_file_path = 'train.csv'
6
7 # we read the content file into a DataFrame called home_data
8 home_data = pd.read_csv('iowa')
9
10 # we try to verify the data by looking at the first few rows
11 print(home_data.head())
```

The Run console shows the output of the script, displaying the first 5 rows of the DataFrame and summary statistics:

	Order	PID	MS	SubClass	...	Sale Type	Sale Condition	SalePrice
0	534	531363010	20	...	WD	Normal	159000	
1	803	906203120	20	...	WD	Normal	271900	
2	956	916176030	20	...	COD	Abnormal	137500	
3	460	528180130	120	...	WD	Normal	248500	
4	487	528290030	80	...	WD	Normal	167000	

Summary statistics (5 rows x 82 columns):

	Order	PID	...	Yr Sold	SalePrice
count	2197.000000	2.197000e+03	...	2197.000000	2197.000000
mean	1460.512517	7.127872e+08	...	2007.793810	182376.851161
std	846.537251	1.887751e+08	...	1.314345	81168.157405
min	1.000000	5.263011e+08	...	2006.000000	13100.000000

Step 2: Review The Data

Use the command you learned to view summary statistics of the data, we will use the .describe() method

The screenshot shows the PyCharm IDE with the same Python script as before, but with an additional line of code added at line 13:

```
13 print(home_data.describe())
```

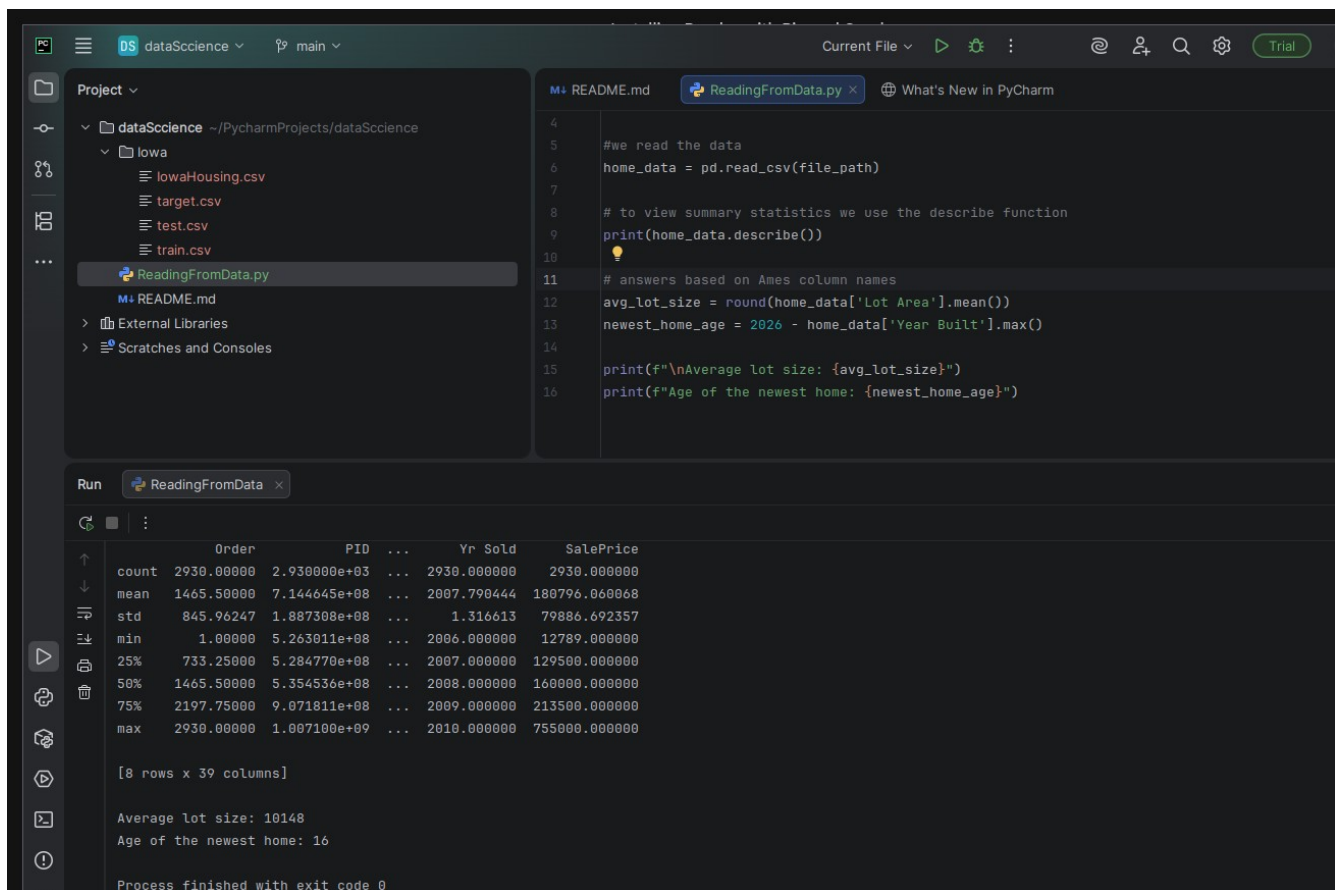
The Run console shows the output of the script, displaying the first 5 rows of the DataFrame and summary statistics:

	Order	PID	...	Yr Sold	SalePrice
count	2197.000000	2.197000e+03	...	2197.000000	2197.000000
mean	1460.512517	7.127872e+08	...	2007.793810	182376.851161
std	846.537251	1.887751e+08	...	1.314345	81168.157405
min	1.000000	5.263011e+08	...	2006.000000	13100.000000
25%	733.000000	5.284410e+08	...	2007.000000	130000.000000
50%	1452.000000	5.354532e+08	...	2008.000000	163500.000000
75%	2196.000000	9.071811e+08	...	2009.000000	215000.000000
max	2929.000000	1.007100e+09	...	2010.000000	755000.000000

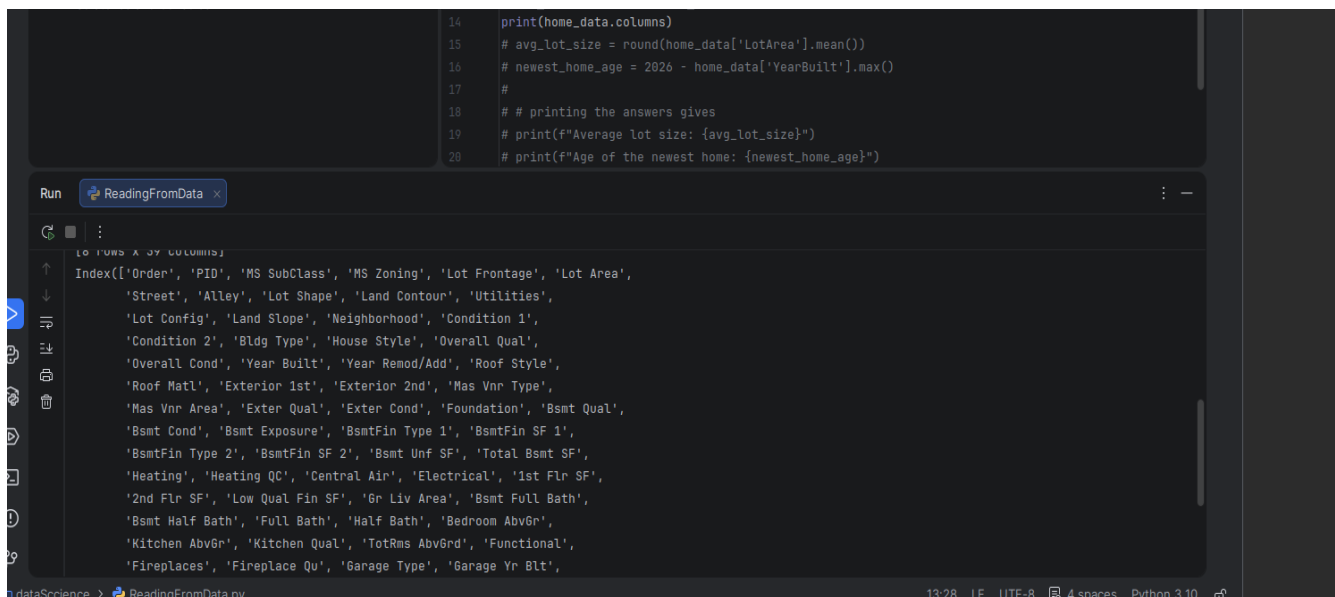
Summary statistics (8 rows x 39 columns):

Process finished with exit code 0

•What is the average lot size (rounded to nearest integer)?



Viewing the number of columns and the names due to typo



- As of today, how old is the newest home (current year - the date in which it was built) and Write out a command to print out your answers: We need to find the maximum value in the Year Built column and subtract it from the current year.

solution

We use `.max()` to find the most recent year

```

17
18 newest_build_year = home_data['Year Built'].max()
19 newest_home_age = 2026 - newest_build_year
20 print(f"The newest home was built in {newest_build_year}.")
21 print(f"As of today (2026), the newest home is {newest_home_age} years old.")

```

Run ReadingFromData x

count	2930.00000	2.930000e+03	...	2930.000000	2930.000000
mean	1465.50000	7.144645e+08	...	2007.790444	180796.060068
std	845.96247	1.887308e+08	...	1.316613	79886.692357
min	1.00000	5.263011e+08	...	2006.000000	12789.000000
25%	733.25000	5.284770e+08	...	2007.000000	129500.000000
50%	1465.50000	5.354536e+08	...	2008.000000	160000.000000
75%	2197.75000	9.071811e+08	...	2009.000000	213500.000000
max	2930.00000	1.007100e+09	...	2010.000000	755000.000000

[8 rows x 39 columns]
The newest home was built in 2010.
As of today (2026), the newest home is 16 years old.
Process finished with exit code 0

Step 3: Think About Your Data (Non-coding):

the reason is explanation 1 They haven't built new houses where this data was collected. Uses the model, but only for that specific area

PHASE 2

Step 1: Specify Prediction Target

In machine learning, the variable we are trying to predict is called the **target** and is named y

```

21 # print(f"As of today (2026), the newest home is {newest_home_age} years old.")
22
23 # we save the sales price to the variable y
24 y = home_data.SalePrice
25
26 # we then display the top few rows to verify
27 print(y.head())

```

Run ReadingFromData x

```

/usr/bin/python3.10 /home/maheva/PycharmProjects/dataScience/ReadingFromData.py
0    215000
1    105000
2    172000
3    244000
4    189900
Name: SalePrice, dtype: int64
Process finished with exit code 0

```

Step 2: Create X

firstly we create a list containing all the provided columns;

create the data-frame X. use that list to pull the specific columns out of the home_data DataFrame

print the result

Project: dataScience ~/PycharmProjects/dataScience

- lowa
 - lowaHousing.csv
 - target.csv
 - test.csv
 - train.csv
 - ReadingFromData.py
 - README.md
- External Libraries
- Scratches and Consoles

```

29 feature_names = [
30     'Lot Area',
31     'Year Built',
32     '1st Flr SF',
33     '2nd Flr SF',
34     'Full Bath',
35     'Bedroom AbvGr',
36     'TotRms AbvGrd'
37 ]
38
39 X = home_data[feature_names]
40 #we print the result
41
42 print("First 5 rows of X:")
43 print(X.head())
44
45 print("\nSummary statistics for X:")
46 print(X.describe())

```

Run: ReadingFromData

```

/usr/bin/python3.10 /home/maheva/PycharmProjects/dataScience/ReadingFromData.py
First 5 rows of X:
   Lot Area  Year Built  1st Flr SF  ...  Full Bath  Bedroom AbvGr  TotRms AbvGrd
0    31770    1960      1656  ...      1          3           7
1    11622    1961       896  ...      1          2           5
2    14267    1958     1329  ...      1          3           6
3    11160    1968     2110  ...      2          3           8
4    13830    1997       928  ...      2          3           6

[5 rows x 7 columns]

Summary statistics for X:
   Lot Area  Year Built  ...  Bedroom AbvGr  TotRms AbvGrd
count  2930.000000  2930.000000  ...  2930.000000  2930.000000
mean   18147.921843  1971.356314  ...    2.854266    6.443003
std    7880.017759    30.245361  ...    0.827731    1.572964
min    1300.000000  1872.000000  ...    0.000000    2.000000
25%    7440.250000  1954.000000  ...    2.000000    5.000000
50%    9436.500000  1973.000000  ...    3.000000    6.000000
75%   11555.250000  2001.000000  ...    3.000000    7.000000
max   215245.000000  2010.000000  ...    8.000000   15.000000

```

Step 3: Review Data (Non-coding)

Before building a model, take a quick look at X to verify it looks sensible.

```

67 # print("\nModel predictions:")
68 # print(predictions[:5])
69
70 from sklearn.model_selection import train_test_split
71 #
72 # # splits data into training and validation data
73 train_X, val_X, train_y, val_y = train_test_split(home_data, target, random_state=1)
74 iowa_model = DecisionTreeRegressor(random_state=1)
75 #
76 # # fit the model with the trained data
77 iowa_model.fit(train_X, train_y)
78 val_predictions = iowa_model.predict(val_X)
79 val_mae = mean_absolute_error(val_y, val_predictions)
80 #
81 # # printing the result
82 print(f"Validation MAE: {val_mae:.2f}")

```

Run: ReadingFromData

```

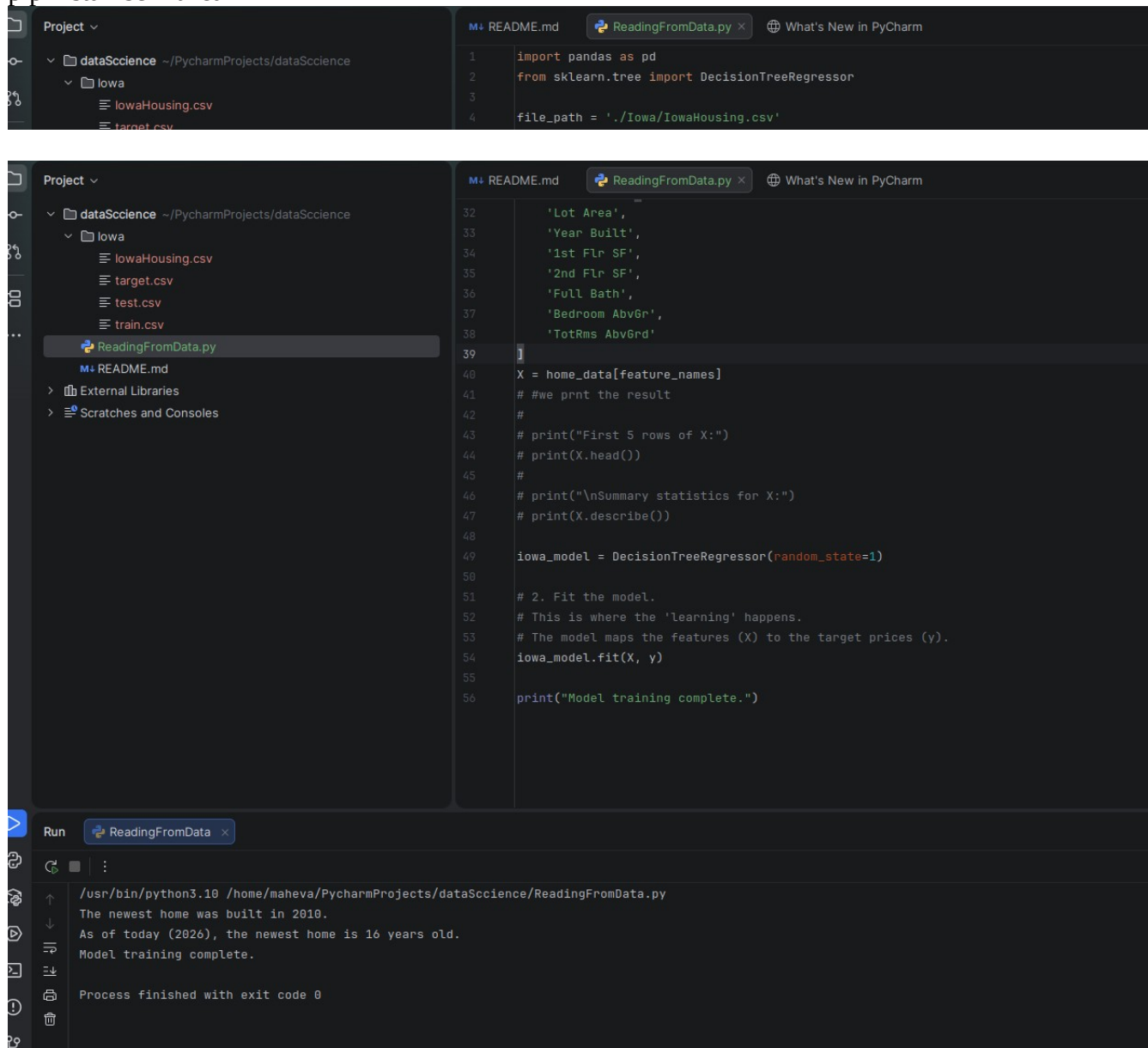
/usr/bin/python3.10 /home/maheva/PycharmProjects/dataScience/ReadingFromData.py
First 5 predictions:
[215000. 105000. 172000. 244000. 189900.]
Actual target values (y):
[215000, 105000, 172000, 244000, 189900]
Validation MAE: $26,439.84

Process finished with exit code 0

```

Step 4: Specify and Fit Model

pip install scikit-learn



Step 5: Make Predictions

Make predictions with the model's predict command using X as the data. Save the results to a variable called predictions.

Now, we have train our model using X and Y data

Generate Predictions

we use the `.predict()` method on `iowa_model`. Since we want to see how the model performs on the data it just learned, we will pass the entire DataFrame X into the command.

```
47 # print(X.describe())
48
49 iowa_model = DecisionTreeRegressor(random_state=1)
50
51 # 2. Fit the model.
52 # This is where the 'learning' happens.
53 # The model maps the features (X) to the target prices (y).
54 iowa_model.fit(X, y)
55
56 print("Model training complete.")
57 predictions = iowa_model.predict(X)
58
59 # printing the first 5 predicted prices
60 print("First 5 predictions:")
61 print(predictions[:5])
62
63 # Compare predictions to actual values
64 print("Actual target values (y):")
65 print(y.head().tolist())
66
67 print("\nModel predictions:")
68 print(predictions[:5])
```

Run ReadingFromData x

```
[215000. 105000. 172000. 244000. 189900.]
Actual target values (y):
[215000, 105000, 172000, 244000, 189900]

Model predictions:
[215000. 105000. 172000. 244000. 189900.]

Process finished with exit code 0
```

Think About Your Results

Use the head method to compare the top few predictions to the actual home values (in y) for those same homes. Anything surprising?

It's natural to ask how accurate the model's predictions will be and how you can improve that. That will be your next step.

Comparing Predictions to Actuals

Since predictions is a NumPy array and y is a Pandas Series, we can use `.head()` for y and a slice

```
67 # print("\nModel predictions:")
68 # print(predictions[:5])
69
70 from sklearn.model_selection import train_test_split
71 #
72 # # splits data into training and validation data
73 train_X, val_X, train_y, val_y = train_test_split(*arrays: X, y, random_state=1)
74 iowa_model = DecisionTreeRegressor(random_state=1)
75 #
76 # # fit the model with the trained data
77 iowa_model.fit(train_X, train_y)
78 val_predictions = iowa_model.predict(val_X)
79 val_mae = mean_absolute_error(val_y, val_predictions)
80 #
81 # # printing the result
82 print(f"Validation MAE: ${val_mae:.2f}")
```

Run ReadingFromData x

```
/usr/bin/python3.10 /home/maheva/PycharmProjects/dataScience/ReadingFromData.py
First 5 predictions:
[215000. 105000. 172000. 244000. 189900.]
Actual target values (y):
[215000, 105000, 172000, 244000, 189900]
Validation MAE: $26,439.84

Process finished with exit code 0
```

for predictions


```
51 iowa_model = DecisionTreeRegressor(random_state=1)
52 #
53 # # The model maps the features (X) to the target prices (y).
54 iowa_model.fit(X, y)
55 #
56 # print("Model training complete.")
57 predictions = iowa_model.predict(X)
58 #
59 # # printing the first 5 predicted prices
60 print("First 5 predictions:")
61 print(predictions[:5])
62 #
63 # # Compare predictions to actual values
64 print("Actual target values (y):")
65 print(y.head().tolist())
66 #
67 # print("\nModel predictions:")
```

Run ReadingFromData x

/usr/bin/python3.10 /home/maheva/PycharmProjects/dataScience/ReadingFromData.py

First 5 predictions:
[215000. 105000. 172000. 244000. 189900.]

Actual target values (y):
[215000, 105000, 172000, 244000, 189900]

Process finished with exit code 0

PHASE 3

Recap

You've built a model. In this exercise you will test how good your model is.

Step 1: Split Your Data

```
67 # print("\nModel predictions:")
68 # print(predictions[:5])
69
70 from sklearn.model_selection import train_test_split
71 #
72 # # splits data into training and validation data
73 train_X, val_X, train_y, val_y = train_test_split(X, y, random_state=1)
74 iowa_model = DecisionTreeRegressor(random_state=1)
75 #
76 # # fit the model with the trained data
77 iowa_model.fit(train_X, train_y)
78 val_predictions = iowa_model.predict(val_X)
79 val_mae = mean_absolute_error(val_y, val_predictions)
80 #
81 # # printing the result
82 print(f"Validation MAE: ${val_mae:,.2f}")
```

Run ReadingFromData x

/usr/bin/python3.10 /home/maheva/PycharmProjects/dataScience/ReadingFromData.py

First 5 predictions:
[215000. 105000. 172000. 244000. 189900.]

Actual target values (y):
[215000, 105000, 172000, 244000, 189900]

Validation MAE: \$26,439.84

Process finished with exit code 0

interpretation: from the result, we see that the predicted prices are no longer identical to the actual prices

In-sample: the model memorizes the answers it was shown during the training

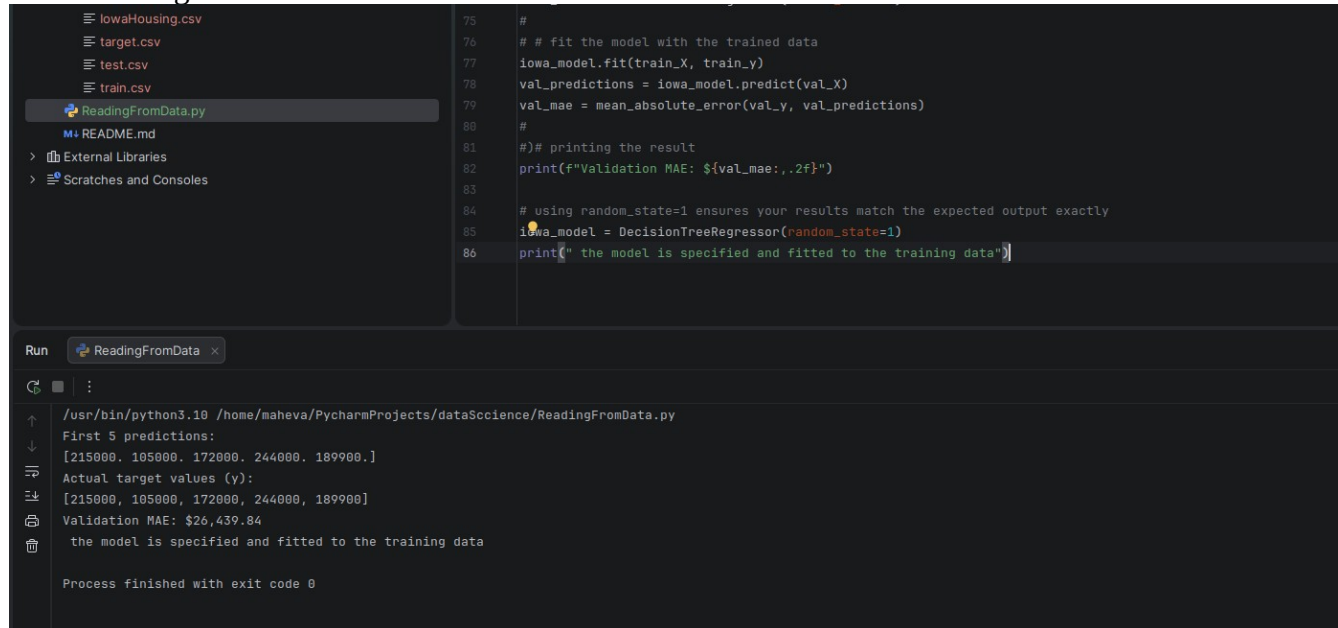
validation: here, the model is guessing based on what it learns but has no idea about that specific house

Validation predictions differ from in-sample predictions because of a concept called **Generalization:**

- **In-Sample (Training) Predictions:** The model is being tested on its "study notes." Since it was built specifically to fit that data. This often leads to **overfitting**, where the model captures random noise in the training data rather than true housing market trends.
- **Validation Predictions:** This is the "real-world test." The model is applying the rules it learned (from our case study example : "more bedrooms = higher price") to new scenarios. The differences we see are the **prediction errors**, which represent how well the model actually understands the relationship between house features and prices.

Step 2: Specify and Fit the Model

Create a DecisionTreeRegressor model and fit it to the relevant data. Set random_state to 1 again when creating the model.



```

75 #
76 # # fit the model with the trained data
77 iowa_model.fit(train_X, train_y)
78 val_predictions = iowa_model.predict(val_X)
79 val_mae = mean_absolute_error(val_y, val_predictions)
80 #
81 #)# printing the result
82 print(f"Validation MAE: ${val_mae:,.2f}")
83
84 # using random_state=1 ensures your results match the expected output exactly
85 iowa_model = DecisionTreeRegressor(random_state=1)
86 print(" the model is specified and fitted to the training data")

```

Run ReadingFromData x

```

/usr/bin/python3.10 /home/maheva/PycharmProjects/dataScience/ReadingFromData.py
First 5 predictions:
[215000. 105000. 172000. 244000. 189900.]
Actual target values (y):
[215000, 105000, 172000, 244000, 189900]
Validation MAE: $26,439.84
 the model is specified and fitted to the training data

Process finished with exit code 0

```

Calculate the Mean Absolute Error in Validation Data

Check if your MAE good? There isn't a general rule for what values are good that applies across applications.


```
77 val_mae = mean_absolute_error(val_y, val_predictions)
78 #
79
80 #)# printing the result
81 print(f"Validation MAE: ${val_mae:,.2f}")
82
83
84 # using random_state=1 ensures your results match the expected output exactly
85 iowa_model = DecisionTreeRegressor(random_state=1)
86 iowa_model.fit(train_X, train_y)
87 val_predictions = iowa_model.predict(val_X)
88 print(" the model is specified and fitted to the training data")
89
90 val_predictions = iowa_model.predict(val_X)
91 print("the top 5 Validation Predictions include:")
92 print(val_predictions[:5])
93 print("\nthe top 5 Actual Prices (Validation Data) are :")
94 print(val_y.head())
```

Run ReadingFromData x

```
/usr/bin/python3.10 /home/maheva/PycharmProjects/dataScience/ReadingFromData.py
First 5 predictions:
[215000. 105000. 172000. 244000. 189900.]
Actual target values (y):
[215000, 105000, 172000, 244000, 189900]
Validation MAE: $26,439.84
 the model is specified and fitted to the training data
the top 5 Validation Predictions include:
[138000. 129500. 202500. 217300. 204750.]

the top 5 Actual Prices (Validation Data) are :
2126 123600
192 209500
2406 202665
45 224000
2477 187000
Name: SalePrice, dtype: int64

Process finished with exit code 0
```

PHASE 4

Recap

You should write the function `get_mae` yourself.

The `get_mae` function

dataScience - /pycharmProjects/dataScience

lowa

lowaHousing.csv

target.csv

test.csv

train.csv

ReadingFromData.py

README.md

External Libraries

Scratches and Consoles

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

```

def get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y):
    model = DecisionTreeRegressor(max_leaf_nodes=max_leaf_nodes, random_state=0)
    model.fit(train_X, train_y)
    preds_val = model.predict(val_X)
    mae = mean_absolute_error(val_y, preds_val)

    return mae

# test 5 leaves
mae_5 = get_mae(max_leaf_nodes=5, train_X, val_X, train_y, val_y)
# test 50 leaves
mae_50 = get_mae(max_leaf_nodes=50, train_X, val_X, train_y, val_y)
# test 100 leaves
mae_100 = get_mae(max_leaf_nodes=100, train_X, val_X, train_y, val_y)
# test 500 leaves
mae_500 = get_mae(max_leaf_nodes=500, train_X, val_X, train_y, val_y)
#print all the leaves and compare
print(f"MAE for 5 leaves: ${mae_5:.0f}")
print(f"MAE for 50 leaves: ${mae_50:.0f}")
print(f"MAE for 100 leaves: ${mae_100:.0f}")
print(f"MAE for 500 leaves: ${mae_500:.0f}")

```

Run

ReadingFromData x

11380000, 1295000, 2025000, 2175000, 2047500]

the top 5 Actual Prices (Validation Data) are :

2126

123600

192

209500

2406

202665

45

224000

2477

187000

Name: SalePrice, dtype: int64

MAE for 5 leaves: \$33,564

MAE for 50 leaves: \$23,501

MAE for 100 leaves: \$23,377

MAE for 500 leaves: \$25,575

Step 2: Fit Model Using All Data

dataScience - /pycharmProjects/dataScience

lowa

lowaHousing.csv

target.csv

test.csv

train.csv

ReadingFromData.py

README.md

External Libraries

Scratches and Consoles

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

```

# def get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y):
#     model = DecisionTreeRegressor(max_leaf_nodes=max_leaf_nodes, random_state=0)
#     model.fit(train_X, train_y)
#     preds_val = model.predict(val_X)
#     mae = mean_absolute_error(val_y, preds_val)
#
#     return mae
# # test 5 leaves
# mae_5 = get_mae(5, train_X, val_X, train_y, val_y)
# # test 50 leaves
# mae_50 = get_mae(50, train_X, val_X, train_y, val_y)
# # test 100 leaves
# mae_100 = get_mae(100, train_X, val_X, train_y, val_y)
# # test 500 leaves
# mae_500 = get_mae(500, train_X, val_X, train_y, val_y)
# #print all the leaves and compare
# print(f"MAE for 5 leaves: ${mae_5:.0f}")
# print(f"MAE for 50 leaves: ${mae_50:.0f}")
# print(f"MAE for 100 leaves: ${mae_100:.0f}")
# print(f"MAE for 500 leaves: ${mae_500:.0f}")

best_tree_size = 100
final_model = DecisionTreeRegressor(max_leaf_nodes=best_tree_size, random_state=1)
final_model.fit(X, y)
print("final model is trained on all data")

```

Run

ReadingFromData x

2406

202665

45

224000

2477

187000

Name: SalePrice, dtype: int64

final model is trained on all data

>

Process finished with exit code 0

PHASE 5:

A single Decision Tree can be very sensitive to small details in the data (overfitting). A Random Forest creates a "forest" of many trees, each looking at the data slightly differently. By averaging their results, it cancels out the errors made by individual trees.

```
119 # final_model = DecisionTreeRegressor(max_leaf_nodes=best_tree_size, random_state=1)
120 # final_model.fit(X, y)
121 # print("final model is trained on all data")
122 from sklearn.ensemble import RandomForestRegressor
123 from sklearn.metrics import mean_absolute_error
124 rf_model = RandomForestRegressor(random_state=1)
125
126 rf_model.fit(train_X, train_y)
127
128 rf_val_predictions = rf_model.predict(val_X)
129 rf_val_mae = mean_absolute_error(val_y, rf_val_predictions)
130
131 print(f"the validation mean average error | for Random Forest: ${rf_val_mae:,.2f}")
```

Run ReadingFromData x

45 224000
2477 187000
Name: SalePrice, dtype: int64
the validation mean average error for Random Forest: \$19,984.94
Process finished with exit code 0